

Abadi Pikosom

Claim: The multiset of total anger levels over all $N!$ orderings has a bijection with the multiset of $C_1A_1 + C_2A_2 + \dots + C_NA_N$ for all possible arrays C satisfying $i \leq C_i \leq N$.

Proof:

Let's say we make an ordering represented by a permutation $[P_1, P_2, \dots, P_N]$ which means that the i -th person who rides the ride is person P_i . The total number of times A_{P_i} is added to the total anger is i added by the number of j such that $j > i$ and $P_j < P_i$.

We will proceed by induction. This claim is clearly true for $N = 1$.

Suppose the claim is true for N , we will prove it for $N + 1$. Suppose we have the ordering $[P_1, P_2, \dots, P_{N+1}]$, and we find the index k such that $P_k = 1$. Let's construct an array $[C_1, C_2, \dots, C_{N+1}]$ which represents that the total anger level is $\sum_{i=1}^{N+1} C_i A_i$. Notice that C_1 (the contribution of A_1) is equal to k .

Next, we claim that for the ordering $[P_1, P_2, \dots, P_{k-1}, P_{k+1}, \dots, P_{N+1}]$, the total anger level is $\sum_{i=2}^{N+1} (C_i - 1) A_i$. It suffices to show that for each i , the value of i added by the number of j such that $j > i$ and $P_j < P_i$ decreases by exactly 1 after removing P_k . If $i < k$, then i remains the same but the number of j such that $j > i$ and $P_j < P_i$ decreases by 1, as k was a possible value of j beforehand. If $i > k$, then the number of j such that $j > i$ and $P_j < P_i$ remains the same but i decreases by 1. ■

Therefore, the problem reduces to finding the K -th largest element of the set

$$\left\{ \sum_{i=1}^N C_i A_i \mid i \leq C_i \leq N \right\}.$$

We proceed with binary search.

If we fix a value V , we want to know solve the following problem:

- Determine if there are at least K possible C such that $\sum_{i=1}^N C_i A_i \geq V$.

Define a function $\text{dfs}(k, \text{val})$ where (k, val) means that we have fixed C_k for $1 \leq k < i$ so that when $C_k = N$ for $i \leq k \leq N$, we have $\sum_{i=1}^N C_i A_i = \text{val}$.

If we go to state $\text{dfs}(k, \text{val})$ where $\text{val} < V$, we will break.

Whenever we reach a state $\text{dfs}(N + 1, \text{val})$, we will add 1 to some global counter of number of possible C .

From $\text{dfs}(k, \text{val})$, we will go to all states $\text{dfs}(k + 1, \text{val} - A_k \times z)$ for $0 \leq z \leq N - k$.

When our global counter of possible C is at least K , we will terminate the algorithm, since we know that there are at least K possible C such that $\sum_{i=1}^N C_i A_i \geq V$.

As such, it is easy to see that the number of calls to $\text{dfs}(k, \text{val})$ that does not immediately return is bounded by at most $O(NK)$.

To optimize this, notice that for many calls to dfs , the only call that actually matters is $\text{dfs}(k + 1, \text{val})$.

This happens when $val - A_k < V$.

To optimize the complexity beyond $O(NK)$, when we are at $\text{dfs}(k, val)$, we will simply call $\text{dfs}(k', val)$ where k' is the smallest index such that $val - A_{k'} \geq V$.

This ensures that the number of calls to dfs is bounded by $O(N)$.

Since each call to dfs requires $O(\log N)$ time, the time complexity of the subroutine is instead bounded by $O(K \log N)$.

Therefore, our final time complexity is $O(K \log N \log \text{MAX})$.

For the more general case, you can see [Eppstein's algorithm](#) to find the k -th shortest path in a graph.

Emang Harusnya Bet Merah

For a set S that's a subset of $\{1, 2, \dots, N\}$, let dp_S denote the expected sum of values in the end for dice in S if the currently unlocked dice are the set S .

Let us consider how to calculate dp_S . For each $i \in S$, let $S_i = S \setminus \{i\}$. After one roll, let's say we get some configuration where the i -th dice rolls on face p_i for all $i \in S$. If we choose to lock the i -th dice, our expected value is $A_{i,p_i} + dp_{S_i}$. That means, for every possible configuration p , we want to calculate the expected value of $\max_i (A_{i,p_i} + dp_{S_i})$.

We will iterate over all $A_{i,p_i} + dp_{S_i}$ when it is the maximum value of our expression. For some i , if $A_{i,p_i} + dp_{S_i}$ is the maximum value, then for all $j \neq i$, the value of p_j must be in a way such that $A_{j,p_j} + dp_{S_j} < A_{i,p_i} + dp_{S_i}$ (let us assume that no two values are exactly the same).

To quickly perform the calculation, let us sort all $|S| \times M$ values of $A_{i,j} + dp_{S_i}$ in increasing order. Then when we sweep over each value, we can store an array `cnt` where `cnti` is the number of times we have already seen $A_{i,j} + dp_{S_i}$ for any j . We use the product of $\frac{\text{cnt}_i}{M}$ to calculate the probabilities.

This allows us to calculate dp_S in $O(|S| \times M \times \log |S|)$. We have the term $\log |S|$ instead of $\log(|S| \times M)$ in our complexity because we can use merge sort on our array, where we have already sorted the values on the faces of each dice beforehand. So we only need to merge $|S|$ sorted arrays of length M together.

Time complexity: $O(2^N NM \log N)$

Note: Less optimal complexities are still allowed to pass for this time limit.

Gegege

Assign the values -1 and 1 to **G** and **E** respectively.

We claim that the minimum number of partitions needed is the maximum absolute differences D between prefix sums.

In the sample **GEGGGEG**, the prefix sums will be $[0, -1, 0, -1, -2, -3, -2, -3]$.

The maximum absolute different is 3 . So at least 3 partitions are required.

This is because we can label each string as a **G**-string or an **E**-string. We can only add **G** and **E** to **G**-string and **E**-string respectively.

If we have a G-strings and b E-strings at the beginning (when the prefix sum is 0), we will have $a + k$ G-strings and $b - k$ E-strings later when the prefix sum is k .

Therefore, if the prefix sums are both p and q and some point where $p - q = D$, we must have $b - p \geq 0$ and $a + q \geq 0$. Therefore, $a + b \geq p - q = D$.

We also know that we require at most D partitions too. We will assign one string to all transitions of the form $i \rightarrow i + 1$ and $i + 1 \rightarrow i$ in the prefix sum.

To count the number of ways to obtain the minimum number of partitions, let p and q be the minimum and maximum values of the prefix sums respectively. We should have p G-strings and q E-strings respectively.

When we encounter a character G, there are p different strings we can append it to. And then, we perform $(p, q) \leftarrow (p - 1, q + 1)$ to account for the fact that we changed a G-string to an E-string.

Similarly, when we encounter a character E, there are q different strings we can append it to. And then, we perform $(p, q) \leftarrow (p + 1, q - 1)$ to account for the fact that we changed an E-string to a G-string.

Do remember to divide the answer by $p!q!$ at the end to account for overcounting.

Ikam Bokam

Add 2.

Jsteyki

Let C_t denote the number of configurations (S, G, L) such that the minimum amount of time to complete the level is t .

Our solution first finds the array C_t in $O(N \log N)$ and then calculates the final answer from array C_t in $O(N \log^2 N)$.

Finding Array C_t

There are 3 cases we should consider in our analysis of the minimum time to complete a level:

- $L = 0$
- $L = 1$
- $L \geq 2$

Suppose we want to go from tile (a, b) to (c, d) .

If $L \geq 2$, the minimum time required is the parity of $a + b + c + d$.

It is easy to show that in at most 2 moves, a bishop can travel between any pairs of tiles that are the same color in an $N \times N$ chessboard.

It is easy to find the contribution of $L \geq 2$ to C .

For $L = 0$, the minimum time is clearly the Manhattan distance, $|a - c| + |b - d|$.

For $L = 1$, consider the transformation $(x, y) \rightarrow (x + y, x - y)$. Then a bishop move can only change at most one coordinate after the transformation.

Therefore, the minimum time is at least $\min(|(a + b) - (c + d)|, |(a - b) - (c - d)|)$.

If we want to find the contribution of $L = 0$ to C only, it is hard to do so. The best complexity we currently have is $O(N^{1.5+o(1)})$.

However, the trick is to find the number of games for both $L = 0$ and $L = 1$ at the same time.

Notice that $|a - c| + |b - d| = \max(|(a + b) - (c + d)|, |(a - b) - (c - d)|)$.

Therefore, we can calculate the contribution of $L = 0$ and $L = 1$ to C_t easily by adding $|(a + b) - (c + d)|$ and $|(a - b) - (c - d)|$ over all pairs of (a, b) and (c, d) respectively to C .

For more details, let $[x^k]A(x)$ be the number of pairs of (a, b) such that $a + b = k$.

Then $[x^k]A(x) \cdot A(-x)$ is the number of pairs (a, b) and (c, d) such that $(a + b) - (c + d) = k$.

Therefore, we will add $[x^k]A(x) \cdot A(-x)$ to $C_{|k|}$.

This allows us to compute the contribution of $L = 0$ and $L = 1$ to C in $O(N \log N)$.

Calculating the Final Answer from Array C_t

Given the array C_t , we will calculate the polynomial $F(x) = \prod_{t \geq 0} (1 + C_t \cdot x)$.

Using DnC FFT, it can be done in $O(N \log^2 N)$. An easy implementation is to place all polynomials in a queue. Then while the queue has size greater than 1, pop the first 2 polynomials and add their convolution to the back of the queue.

Clearly $[x^k]F(x)$ is the number of ways to choose k unordered levels so that all k levels have different minimum completion time.

So the answer is $\sum_{k \geq 0} k! \cdot [x^k]F(x)$.

Lama Pute Pute

Let $g(x)$ denote the maximum k possible when we can rearrange the dancers from 1 to x .

For a fixed set of dancers who are on slots $[x_1, x_2, \dots, x_l]$ such that $x_1 < x_2 < \dots < x_l$, it is clear that the maximum value of k achievable is $\gcd(x_2 - x_1, x_3 - x_2, \dots, x_l - x_{l-1})$. Next, if we've found the maximum k possible (let's say it's m), then all possible values of k for this set of dancers are all factors of m .

Let $d(x)$ be the number of factors of x for convenience. Consider the sequence $[g(N - 2), g(N - 1), \dots, g(1)]$. Notice that $g(i)$ is always a factor of $g(i + 1)$, because adding a new dancer with a fixed position is like adding a new integer to the set of numbers whose GCD we want to calculate. We want to maximize $d(g(N - 2)) + d(g(N - 1)) + \dots + d(g(1))$.

We will do dp on the distinct values of g in our sequence. Specifically, suppose $[b_1, b_2, \dots, b_k]$ is the sequence such that b_i is a multiple of b_{i+1} and b is the unique elements of g in our sequence. Note that $d(b_1) > d(b_2) > \dots > d(b_k)$.

It turns out that for the best strategy to maximize the final answer according to our array b , we iterate the dancers from N to 1, and we start placing them in every slot whose index is 1 plus a multiple of b_1 , then we start placing them in every slot whose index is 1 plus a multiple of b_2 , and so on.

This strategy is optimal because the upper bound on the number of values of g who is a multiple of b_1 is $\lceil \frac{N}{b_1} \rceil - 1$, the upper bound on the number of values of g who is a multiple of b_2 is $\lceil \frac{N}{b_2} \rceil - 1$, and so on. And it is clear that our construction achieves this upper bound. Note that in our construction, it is possible that the value of g is not exactly b_i but instead a multiple of b_i .

To solve the problem, we will perform dynamic programming on the array b . Specifically, dp_i denotes the maximum score we can achieve after placing all dancers whose positions are 1 plus a multiple of i . Then the value of dp_i is the maximum value of $dp_j + d(i) \times (\lceil \frac{N}{i} \rceil - \lceil \frac{N}{j} \rceil)$ for all j that's a multiple of i . Our answer will be dp_1 .

The number of pairs of integers $1 \leq i, j \leq N$ such that i divides j is $O(N \log N)$. Therefore, the DP can be calculated in $O(N \log N)$.

Like a Comet

It turns out that if it is possible to not push any disappointing episode, the optimal solution is to make excellent episodes whenever possible, and here is the proof.

Assume there is an order of episode quality so there are nonzero excellent episode not being pushed out even though it is possible.

Observe the excellent episode, with the biggest number of adjacent ready episode before it, and if it is tied, choose the larger indexed one. Assume that is the i -th episode, while being able to be pushed on the $(i - 1)$ -th episode instead and that episode being only a ready episode. Claim that pushing the excellent episode one episode earlier will never lower the total rating.

Assume the rating of the $(i - 1)$ -th episode is x . Right after the i -th episode there must be a sequence of adjacent ready episodes with nonnegative length, and right after that it is either an excellent episode or the end of all episodes. Assume that the last ready episode rating in that adjacent episodes is y . Observe that if $y \leq x$, then the first assumption is false, since the excellent episode should have the biggest number of adjacent ready episode before it and the latest episode if tied. So we have $y > x$

It can be easily observed that switching the excellent episode to the one episode before it will add the total rating by $\max(y - 1, 0) - x$ which is never less than 0. In that case there is a transformation sequence for any order of episode quality, to the maximum total rating of "make excellent episodes whenever possible".

Limas Agung

The most important thing to notice is that the condition $S_{P_i} - S_i \leq S_{P_{P_i}} - S_{P_i}$ means that if we have a path of length D such that top of the path has value A and the bottom of the path has value B , then second top value should have value at most $A - \lceil \frac{A-B}{D} \rceil$. This is because the difference array of the values of the path has to be non-decreasing and sum to $A - B$. So it is clear that the last value has to be at least $\lceil \frac{A-B}{L} \rceil$.

Therefore, this gives a greedy strategy to assign values starting from the root to the leaves.

For each vertex, we will calculate the depth D of the shallowest leaf from it. Then if it's parent has value V , the node will have value $V - \lceil \frac{V-L}{D+1} \rceil$.

This is optimal because the value of each node is dependent on the value of the parent. A larger parent value leads to a larger node value. Since we have maximized the value of the parent, we will also maximize the value of the node itself.

Sarjana Excel

Let's say the width of the rows are $[r_1, r_2, \dots, r_N]$ and the width of the columns are $[c_1, c_2, c_3]$. Notice that if you have a valid configuration of row and column widths, you can always choose an arbitrary multiplier m , then multiply every value r_i by m and divide every value of c_i by m , then you'll get another valid configuration with the same area. That means, you can always turn any valid configuration into a configuration where $c_1 = 1$. From now on, let's solve the problem if we must ensure $c_1 = 1$.

Let's say we've fixed the values of c_2 and c_3 . Consider some row i . Its minimum valid row width is $\max(X_i, \frac{Y_i}{c_2}, \frac{Z_i}{c_3})$. That means, row i will contribute an area of $\max(X_i, \frac{Y_i}{c_2}, \frac{Z_i}{c_3}) \times (1 + c_2 + c_3)$, and the total area is the sum of this value for every row.

For each row i , consider the these three different two-variable functions:

- $f_x(c_2, c_3) = X_i(1 + c_2 + c_3)$
- $f_y(c_2, c_3) = \frac{Y_i}{c_2}(1 + c_2 + c_3) = Y_i(1 + \frac{1+c_3}{c_2})$
- $f_z(c_2, c_3) = \frac{Z_i}{c_3}(1 + c_2 + c_3) = Z_i(1 + \frac{1+c_2}{c_3})$

We can prove that each of those three functions is convex in the 2D function sense. That is, if we draw any straight line on the (c_2, c_3) plane and graph the value of the function for all pairs (c_2, c_3) along that line, we get a convex graph.

Next, notice that the contribution for each row, based on the values of c_2 and c_3 , is the maximum of those three functions. The maximum of convex functions will always result in another convex function.

If we combine the contribution of all rows by summing all row functions (each function is convex), we'll get another convex function. This means, we can solve this by doing a ternary search to find the optimal the value of c_2 , and for each iteration of c_2 , we do another ternary search to find the optimal value of c_3 for that fixed value of c_2 .

However, if we naively do an $O(N)$ computation each time we compute the total area, the total time complexity would be $O(N \log^2 \text{MAX})$ with bad constant factors. We need to do something faster.

The trick is to do precomputations by building a data structure so we can compute the area quickly each time.

Consider the function for the area contribution for a single row i . If we plot the function values on 3D space and we see an overhead view with the c_2 and c_3 axes being the horizontal and vertical axes respectively, we can see that the plane is divided into three regions, each region is for which value among f_x , f_y , and f_z is the biggest. We can calculate that the borders for the division of regions look like this.

If we sum all N functions, we get a combination of the aforementioned borders.

For each row, we precompute these three values:

- $\frac{Y_i}{X_i}$: the c_2 -coordinate of its vertical border.
- $\frac{Z_i}{X_i}$: the c_3 -coordinate of its horizontal border.
- $\frac{Z_i}{Y_i}$: the slope of the diagonal border.

Consider some point (c_2, c_3) and we want to calculate the sum of values for every row function. Divide the rows based on if $\frac{Y_i}{X_i} \leq c_2$ or $\frac{Y_i}{X_i} > c_2$. Call these the left group and the right group.

For rows in the left group, which region it uses is only determined by its horizontal border. So for every such row with $\frac{Z_i}{X_i} \leq c_3$, we use its f_x value. And for the rest within this group, we use its f_z value.

For rows in the right group, which region it uses is only determined by its diagonal border. So for every such row with $\frac{Z_i}{Y_i} \leq \frac{c_3}{c_2}$, we use its f_y value. And for the rest within this group, we use its f_z value.

Notice that both cases only uses two values. That means, this can be solved by handling 2D rectangle sum queries. We can sort the rows by $\frac{Y_i}{X_i}$. Then, we make two merge sort trees, each for the values of $\frac{Z_i}{X_i}$ and for the values of $\frac{Z_i}{Y_i}$. Building each of these segment trees only takes $O(N \log N)$.

When querying a point (c_2, c_3) , we do four rectangle queries. For each sum of f_x , f_y , or f_z that we want to obtain, we only need to calculate the sum of X_i , Y_i , or Z_i respectively, and then we plug the values of c_2 and c_3 into the corresponding function formula.

That means, for each segment tree node, we need to make prefix sums of X_i , Y_i , and Z_i separately following the merge-sorted values. Then for each query, we visit $O(\log N)$ segment tree nodes and to a binary search in its array to get the prefix sum value.

Time complexity: $O(N \log N + N \log N \log^2 \text{MAX})$

Note: Unfortunately, it turns out that it's possible to make the naive solution have small enough constant factors to pass the time limit using optimizations such as golden ratio search.

Shuttekka Panorama

Let $\text{solve}(L, R)$ be a function that updates all answers for ranges $[l, r]$ such that $L \leq l \leq r \leq R$.

Let m be the index with the maximum value in $[L, R]$.

Then for all $L \leq l \leq m \leq r \leq R$, the position of the maximum value will be in m . The position of the tallest mountain is $m - l$, which only depends on l .

Let cnt_k denote the number of ranges such that the highest mountain is in index k .

Then when we process all ranges $L \leq l \leq m \leq r \leq R$, we should add $R - m + 1$ to all $\text{cnt}_0, \text{cnt}_1, \dots, \text{cnt}_{m-L+1}$.

To speed this up, we should instead add $R - m + 1$ to cnt_{m-L+1} and perform a suffix sum at the end.

We still need to process ranges with $r < m$ and $l < m$. But we should call $\text{solve}(L, m - 1)$ and $\text{solve}(m + 1, R)$.

Each call to solve takes $O(1)$ time as we can find m in $O(1)$ using a sparse table. And it is easy to see that solve gets called $O(N)$ times.

By construction a sparse table in $O(N)$, we get a complexity of $O(N)$.

Uau Aiai

It is easy to see that the truck driver needs to complete the deliveries of all the locations in one city and only then move to the next city, same with delivering all cities in one country and only then move to the next country.

Notice that only on the location-by-location level that there are no restriction on visiting the same location more than once. This means the distance between any pair of locations within a city can be Floyd-Warshall-ed with a complexity of $O(N_2^3)$.

We can then process a bitmask dynamic programming to calculate the minimum cost of traversing all locations in one city starting from some location i and ending in some location j , where (i, j) can vary between all pairs of locations (not necessarily different) in that particular city. This can be done in $O(2^{N_2} N_2^2)$ time for each of the $O(N_2)$ starting locations.

Now for the city-level movements. In a single country, for every pair of starting city i and ending city j , we can calculate the minimum silver coin cost to visit every single city in that country in $O(2^{N_1} N_1^2)$ for each of the $O(N_1)$ starting cities.

For each pair (i, j) , we can obtain the optimal path of cities. Notice that because of the special property of g , there is only one optimal path. After getting the optimal path, we can use the order of cities in that path to calculate the minimum bronze coin cost for that minimum silver coin cost using simple dynamic programming connecting all the possible last-location-in-previous-city to first-location-in-next-city. We calculate this for every pair of starting location in city i and ending location in city j . We can do this in $O(N_1 N_2^2)$ time for each of the $O(N_1^2)$ pairs of (i, j) and each of the $O(N_2)$ starting locations. This will result in every country having a minimum silver and bronze cost for every (starting location, ending location) pair.

Before connecting between countries, we can precompute the order of countries visited by doing the bitmask dynamic programming on the highest level starting from any country for $O(2^{N_0} N_0^2)$ time.

We can obtain the optimal path of countries. Just like the cities, we use the special property of f to ensure that there's only one optimal path. We use the order of countries in that path to calculate the minimum silver and bronze pair using simple dynamic programming by connecting the last-location-in-previous-country to first-location-in-next country, in $O(N_0 N_1^2 N_2^2)$ time. We are done.

Time complexity: $O(2^{N_0} N_0^2 + 2^{N_1} N_0 N_1^3 + 2^{N_2} N_0 N_1 N_2^3 + N_0^2 N_1^2 N_2^2 + N_0 N_1^3 N_2^3)$

Ynoalgoget

Consider the string formed by adding **L** when we move to the leftmost character and **R** when we move to the next character.

For convinience, we will let the string start with the character **L**.

Then the process terminates when we encounter $N - 1$ consecutive **R** characters.

Consider the binary strings beginning in **L** that produces the string S at the end. We will call them good strings.

For $S = \text{BABAB}$, we have:

- **LRRRR** (BABAB)
- **LRLRR** (BABAB)
- **LRRRL** (BABAB)

- LRLRL (BABAB)
- LRRRLRR (BABABAB)
- LRRRLRL (BABABAB)

Alternatively, we can think about each binary strings as an array of prefix lengths $[p_1, p_2, \dots, p_k]$ of the main string such that $p_1 + p_2 + \dots + p_k \geq N$, $p_2 + \dots + p_k < N$ and S is also a suffix of $S_{p_1} + S_{p_2} + \dots + S_{p_k}$. In this editorial, S_i denotes $S[1, i]$.

For example, the binary strings above correspond the following prefix arrays:

- $p = [5]$
- $p = [2, 3]$
- $p = [4, 1]$
- $p = [2, 2, 1]$
- $p = [4, 3]$
- $p = [4, 2, 1]$

Using linearity of expectation, we only need to find the expected number of times each good string Z appears in our final binary string.

But this is easy. Again by linearity of expectation,

$$E(\text{occurrence of } P) = E(P \text{ occurs starting from first L}) + E(P \text{ occurs starting from second L}) + \dots$$

We should note that $E(P \text{ occurs starting from } i\text{-th L}) = P(i\text{-th L exists}) \cdot 2^{-|Z|+1}$.

To calculate $P(i\text{-th L exists})$, we note that $P(\text{first L exists}) = 1$ and $P((i+1)\text{-th L exists}) = P(i\text{-th L exists}) \cdot (1 - 2^{-N+1})$.

Therefore, $E(\text{occurrence of } P) = (1 + p + p^2 + \dots) \cdot 2^{-|Z|+1}$, where $p = 1 - 2^{-N+1}$.

Simplifying the above expression, we have $E(\text{occurrence of } P) = 2^{N-|Z|}$.

Therefore, the problem reduces to calculating $\sum 2^{N-|Z|}$ over all good strings.

We will calculate this value using the p array defined earlier.

We will initialize dp_i according to the first element of p .

This means, initially $dp_i = \sum_{S_i \text{ is suffix of } S_j} 2^{i-j}$. Here, we use the coefficient 2^{i-j} to account for the fact that a good string of length $|Z|$ contributes $\sum 2^{N-|Z|}$ to the answer.

To find the set of such j for each i , we will use the KMP function π . Because the set of such j for each i are such that $\pi^{(k)}(j) = i$ for some k . If we build a tree with edge $i \rightarrow \pi(i)$, then the set of such j would be the subtree rooted at vertex i .

Now, the transition of our dp will be dp_i will be added to dp_j if $S[i+1, j]$ is a prefix of S .

It should be obvious that for a fixed i , we can find the largest j such that $S[i+1, j]$ is a prefix of S and then add dp_i to all $dp[i+1, j]$. If we have calculated the Z-function earlier, finding such j is easy.

It seems like we would need a segment tree to handle updates, but this can actually be handled using a prefix sum.

The final answer will be dp_n .

Note that this method is deterministic and linear.